

A Study on the Effect of Target Object Size in Small Object Detection

Jacob Cassar Ellis

ABSTRACT

Recent years have seen an impressive increase in the efficiency and accuracy of object detection models through the use of Region-based Convolutional Neural Networks (RCNN). Many studies have been carried out with respect to testing the limits of object detection, however the detection of small objects through aerial imagery still poses numerous issues. In fact, relatively coarse feature maps lead to the decrease in accuracy of these systems as they lack the information required for sufficient detection. Moreover, accuracy diminishes as these networks struggle to distinguish between foreground and complex backgrounds such as rough terrain.

Keywords

Convolutional Neural Network; object detection; small object;

1. INTRODUCTION

1.1 Problem Definition

Object detection is a task that deals with detecting instances of objects of a specific class. Over the last six years, many researchers have invested their time into developing solutions and improving upon them continuously to provide the most accurate and efficient model [1, 2, 3, 4]. Recently, through the rapid development in deep learning, there have been numerous amounts of breakthroughs in large scale object detection and image classification. Deep Learning has brought about models such as region-based Convolutional Neural Networks that can perform object detection in fast and efficient manner [5].

These models have proven to work extremely well for detection large objects like planes or cars but suffer when attempting to handle small objects. There are several reasons for this which include occlusion, lack of light, and the information present in the image itself. As the size of the object decreases, the spatial resolution between the pixels is reduced, thus resulting in a lack of information to produce a rich representation of the target object [2]. Without an efficient representation, the model is unable to detect the object, or could produce false positives by suggesting the target object is elsewhere. The limits of each model can be tested by varying the distance between the capture device and the target object as it mimics the object becoming smaller. Through this experiment, it will be possible to de-

termine the minimum number of pixels required to correctly detect and object and improve upon it.

1.2 Motivation

The use for small object detection can have many positive impacts in the world of computer vision. In fact, this study was conducted to provide new insights on the limits of current models to help develop a litter-based detection system. The issue of small object detection is at the forefront of such a system as the class of litter varies significantly. Moreover, litter itself comes in all shapes and sizes, thus one can never be certain if there is enough spatial resolution to detect the object. Through this experiment, the spatial resolution limit can be found and used to propose the optimum height required for such a system to be viable in urban and rural areas.

1.3 Why the problem is non-trivial

All modern solutions for object detection are heavily based on Convolutional Neural Networks. These powerful networks operate by extracting low level abstractions from images and combines them to the object that is required to be detected. This occurs between the hidden layers that are present within the CNN, through combinations of convolutions and pooling. Moreover, the recent developments of YOLO, R-CNN and Fast R-CNN's have proven to provide very accurate results in a short amount of time for detecting large objects.

However, the low-level abstractions are the reason that detecting small objects poses such an issue. Current models reduce image resolution between the convolutional layers in order to extract the features of an object. These features are passed into the following layer and eventually combined to create a rich representation of the object. Small objects, however, only take up a small percentage of the image's spatial resolution. Thus, when the resolution is decreased, the small objects located in the image have too little information for the CNN to provide any form of features. This results in the loss of the small object completely.

Current studies have not investigated the limits of these models in terms of the spatial resolution required to detect a small object. Hence, the information provided through this experimental could provide new insights to help develop more accurate systems. Moreover, the results could also be used to pinpoint the limitations of current state-of-the-art models.

2. AIMS AND OBJECTIVES

The aim of this project is to investigate the impact of spatial resolution in small object detection by varying the distance between target object and capture device. In order to achieve this goal, the following objectives have been identified:

- To design and build a dataset for aerial small object detection based on the COCO dataset
- Research multiple Deep Learning networks to obtain the most accurate and time-efficient model for the detection of small objects
- To investigate the accuracy of the chosen models with respect to the performance of small object detection on plain and complex backgrounds
- Deploying a drone-based system to evaluate the limits of the chosen models

3. BACKGROUND RESEARCH

Artificial Neural Networks

The foundation of an Artificial Neural Network (ANN) is based on the biological representation of the human brain. ANN's consist of two fundamental components, the Neurons and the Synapses in order to perform pattern recognition through machine learning [5]. The first form of artificial Neuron created was implemented as a switch that receives an input via other neurons and produces results of which are dependent on the total weighted input designed by McCulloch and Pitts [6]. In addition, the weights presented below represent the strength of the synapse.

The accuracy of the network is dependent on the algorithm used. Initially, Minsky and Papert showed that these networks could only solve linearly separable problems. Although, through the error back-propagation algorithm, ANN's gained the ability to solve non-linearly separable problems whilst increasing their overall accuracy and efficiency [6]. These algorithms allow ANN's to fine tune their parameters with each example that is fed to it, thus increasing its accuracy.

The ANN itself is comprised of multiple layers, each consisting of their own set of Artificial Neurons which are interconnected together to form the network. The first layer found is the input layer, which is where data is fed into the network. The result of this layer is passed through a set of hidden layers. The number of hidden layers found within an ANN are dependent on the form of image classification required. Their role is to capture more complex information in order to create a rich representation of the features of the object being classified. However, many hidden layers lead to over-fitting as relationships that do not exist, are created. Finally, the output layer collects the data provided by the hidden layers and performs the final classification. This multi-layered approach to a neural network is referred to as a feed-forward neural network as seen in Figure 1 [7].

Deep Learning in Neural Networks

As described in the previous sub-section concerning ANN's, these networks learn through the representations formed

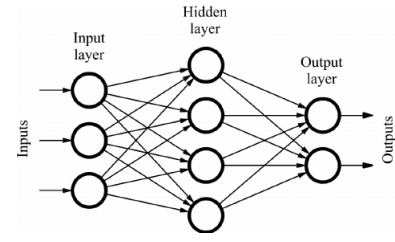


Figure 1: Feed-Forward Neural Network

within the hidden layers. However, these neural networks were not considered deep as data did not have to pass through multiple layers in order to provide some form of classification. This limited what ANN's could achieve, leading to the birth of DANNs [8].

Deep learning can be seen as an extension of ANNs [9] as they make use of the same architecture seen in Figure 1. The depth of these DANNs are heavily dependant on the hardware they are executed on. This is why previous neural networks made use of a single hidden layer, as the hardware that was available at the time inhibited the networks ability to train quickly.

However, the recent developments in hardware have led to networks which can contain tens of hidden layers. These advancements have reduced the time required for a deep learning model to learn exceptionally, from requiring months to as little as weeks depending on the model.

Convolutional Neural Networks

One of the earliest CNNs created was pioneered by Yann LeCun. The neural network, called LeNet5, brought about the use of applying convolutions with learning parameters to an image to extract features within the image [1]. However, GPUs and CPUs were not capable of providing efficient speeds in terms of training. These limitations brought about a drought in research behind CNNs until 2012.

The CNNs that are familiar today are based on the work of Alex Krizhevsky in [2]. The network, called AlexNet was built on the same architecture as [1]. However, made use of the advancements in hardware to achieve exceptional results in image classification on the ImageNet database. Krizhevsky's work brought about the rapidly increasing interest in object detection through CNNs.

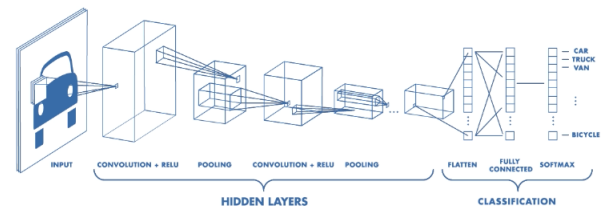


Figure 2: AlexNet [2] CNN Architecture

The architecture seen in Figure 2 makes use of an input layer, convolutional layer or hidden layer, and a classification layer. The input layer of the feed forward neural network described in Figure 1 required the number of neurons the

first layer to be equal to the size of the input. However, for large images, this simple implementation does not scale very well.

CNNs make use of convolutional layers, which give the network the ability to break down full scale images using fewer connections and parameters. A sliding filter is applied to the input, applying convolutions in order to extract a feature map. By applying a convolutional filter in this manner, the spatial information between the pixels is preserved, allowing it to work on large images without incurring operational costs. The filters in each convolutional layer are different. Initially, filters are likely to extract geometric shapes. With each additional convolutional layer, the features extracted become more abstract and complex, which allows us to classify them.

However, before passing the feature map from one convolutional layer to the next, some form of activation function must be applied. In [2], *ReLU* was implemented and is still widely used to this day. Through the work of Nair and Hinton, *ReLU* [3] is used to introduce non-linearity into the CNN. Moreover, this activation function is computational inexpensive thus drastically decreasing the amount of time required to train.

Once the activation function has been applied, the feature map is passed through the pooling layer. This is a form of compression which reduces the number of parameters required by down samples the contents of the feature map whilst retaining spatial information. Some forms of pooling are:

- *Max Pooling*: Works by obtaining the largest value within a given matrix of $n \times n$ pixels. This method is used to depict that a feature is present within the segment being evaluated.
- *Global Average Pooling*: First introduced by [4], GAP Layers are used to reduce the dimension of the feature map whilst retaining spatial information in a more extreme reduction. Moreover, in 2016, GAP was found to aid in object localization as seen in [10].

As depicted in *Figure 2* this process is repeated, with each layer obtained more abstract features the deeper the network is. Moreover, in order to provide some form of classification, these feature maps are passed through the classification layer, into a fully connected network. The matrix of feature maps are flattened to a single vector of values where each value represents the probability for that feature to belong to a specific label. The fully connected neural network also makes use of functions such as dropout to minimize over-fitting whilst the probabilities are generated via a softmax implementation.

4. LITERATURE REVIEW

The previous chapter discussed the main components of neural networks. Moreover, provided an insight to how the Convolutional Neural Networks we know today, are built and based on components that were implemented as far back as the 1940s.

The following chapter will delve deeper into the history of Convolutional Neural Networks, whilst also exploring the branches that have been developed since AlexNet [2]. The additional branches discussed heavily focuses on object detection and will be split into two main sub-sections. The first

will focus on Region-Based Frameworks such as R-CNN [11], while the second will focus on Regression and Classification Based Frameworks like YOLO [12].

Region Proposal Based Framework

The idea behind Region Proposal Based models is extracted from the understanding of the human brain's form of object detection. The process makes use of two stages which involves applying a coarse scan over the full inputted image and then focuses on the regions of interest provided. These ROIs are obtained through the use of a selective search (SS) algorithm [13]. Each pixel within the image is initially considered as an individual group, merging each group of the with similar textures. Once they have been merged, bounding boxes are applied to extract the ROIs.

4.1 R-CNN

Proposed by Gershick in 2014, R-CNN [11] provided significant improvements over the the state-of-the-art model [14] at that time on the PASCAL VOC 12 dataset. R-CNN makes use of the previously explained selective search algorithm to generate around 2000 region proposals for the inputted image. The use of the SS algorithm significantly reduces the searching space in object detection. However, suffers in proposing regions of interest where one object is occluded by another object.

The feature extraction methodology applied in R-CNN makes use of a warping function cropping each region of interest to a fixed resolution. The AlexNet architecture was modified for the CNN module of the network to provide 4096 dimensional features. Once the features are extracted, class specified linear Support Vector Machines (SVM) are used to classify and locate the objects within the image.

Although this model provided some exceptional results at the time, there were some disadvantages that limited its potential. As previously mentioned, the selective search algorithm suffers in the detection of occluded objects. In addition to this, SS is also a time-consuming process as some region proposals are redundant. Moreover, training an R-CNN is a multi-stage pipeline as the CNN module itself must be fine-tuned along with the SVMs. Furthermore, the features extracting using the CNN module are stored on disk. Therefore, a deep CNN module will require a relatively long time to process and a large amount of memory to store the features for the SVMs [15]. Lastly, the requirement of the input image being a specified dimension could lead to a reduction in recognition accuracy. If the input image does not match the correct dimensions, some form of cropping or warping is applied to the image. The use of cropping could lead to cropping the object out of the specified region, whilst warping may cause unwanted geometric shapes to present themselves [16, 17].

4.2 SPP-Net

A solution to the previous mentioned issue of cropping and warping was implemented by He et al. in [16]. SPP itself had been created on the basis of the Bag-Of-Words approach, however was never utilized for Object Detection. He et al. proposed replacing the final pooling layer with a Spatial Pyramid Pooling (SPP) layer in order to address the issue of different image dimensions.

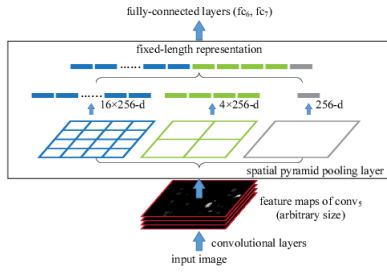


Figure 3: Spatial Pyramid Pooling Implementation [16]

In *Figure 3*, an overview of SPP is provided as depicted in [16]. Spatial Pyramid Pooling allows for the retainment of spatial information by pooling it in local spatial bins. The number of spatial bins is proportional to the size of the image, allowing for any sized input image to be used. The spatial pyramid pooling layer itself makes use of the previously explained max pooling in order to pool the response of each filter of the final convolutional layer. This outputs a set of kM -Dimensional vectors where k is the number of filters in the final convolutional layer.

The fixed-length representation seen in *Figure 3* takes the vector output from each resultant SPP layer. This multi-level pooling feature allows for the finer features to be presented first, whilst the final vector represents the coarsest feature which covers the entire input image. This is also known as "global pooling" which was previously explained in Chapter 2. By implementing the pooling layer in this manner, the feature maps are only computed once. Previously, the feature maps were computed after each convolutional layer, drastically increasing run-time.

In addition, the use of a SPP layer not only decreases testing time, but also decreases training time. This is due to allowing the training images to also have varying dimensions. Allowing for a reduction in over-fitting whilst also increasing scale invariance. This method yielded a drastic increase in speed, by computing features at a rate of 24-104x faster than R-CNN [16].

4.3 Fast R-CNN

Although SPP-Net saw some improvements in accuracy and speed, the remaining issues explained previously in R-CNNs are still present in both models. He et al. only focused on reducing the repetitive nature of feature map computations. Thus, the fine-tuning of the network itself, along with SVM training, feature extraction and bounding box regressor fitting issues were still present [17, 15].

Gershick himself modified his previous R-CNN by applying a simple multi-task loss function to both classification and bounding-box regression in [15] called *Fast R-CNN*. Moreover, applied the modification presented in *SPP-Net*, by processing the whole image using a CNN to produce a convolutional feature map. Using this feature map, region proposals are extracted and applied to the RoI pooling layer. This layer is a special case of SPP as it is a one level pyramid [17] which is used to transform the feature vectors into a fixed size to be fed to the fully connected layer.

Once the feature vectors have passed through the fully connected layers, they are branched into two sibling output layers as seen in *Figure 4*, and trained using a multi-loss

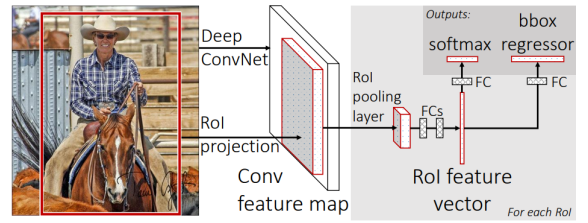


Figure 4: Fast R-CNN Architecture [15]

function. The softmax branch is used to generate a probability distribution over $k + 1$ classes, where k refers to object classes with the addition of a background class. The bounding box regressor branch refines the position of each box using a set of four real-valued numbers for each of the K object classes [15].

4.4 Faster R-CNN

The work mentioned in *SPP-Net* and *Fast R-CNN* sought to reduce the overall execution time for general object detection. By doing this, the region proposal system implemented by both network proved to be a bottleneck [18] as they generally relied on selective search or EdgeBox. Selective search requires around two seconds on CPU implementation per image, whilst EdgeBox requires 0.2 seconds per image, the best at the time.

In order to improve this, Ren et al. in [18] proposed a RPN which allows for cost-free region proposals. This innovation is similar to a fully convolutional network which can be trained in an end to end manner for generating region proposals.

Similar to previous works, Faster R-CNN is a two stage detector. Ren et al introduces the first stage of their model as the RPN, or deep FCN that is used to propose regions. The second stage makes use of the previously explained *Fast R-CNN* design. These two stages are joined together to form a single unified network [15].

The innovation of RPN allows for an image input of varying size to produce a set of object proposals each with their respective score. The set of object proposals is created by sliding an $n \times n$ window over the convolutional feature map returned by the final convolutional layer. By applying this sliding window, a low dimensional feature vector is obtained for each iteration. Moreover, *ReLU* is applied to introduce non-linearity as previously explained. This vector is then fed into two sibling layers, similar to *Fast R-CNN* for box-classification and box-regression, using a modified multi-loss function to produce the appropriate output.

4.5 Mask R-CNN, R-FCN, FPN

Following the work of Ren et al, three models emerged from the use of Region Proposal Networks for object detection. The models being Region-based Fully Convolutional Networks by Dai et al [19], Feature Pyramid Networks by Lin et al [20] and finally, Mask R-CNN by He et al [21].

R-FCN

In [19], an investigation was carried out on the state-of-the-art image classification models that were fully convolution, such as Residual Nets and GoogLeNets. Through this investigation it was found that the network itself has superior

classification accuracy, but lacked the same amount of accuracy for object detection related issues. Moreover, the authors believed that this is *"caused by a dilemma of increasing translation invariance for image classification vs. respecting translation variance for object detection [19]"*. In other words, a change in position of an object within an image should have no adverse effects in classification, however may pose meaningful changes in terms of object detection.

Dai et al proposed a Region-based Fully Convolutional Network attempts to handle the issue of translation variance in object detection against translation invariance in image classification. The modification applied concerns the final convolution layer of the R-FCN. This final layer produces a k^2 position-sensitive score maps for each category using a $k \times k$ grid. Each of these score maps contains information on the position of the object relative to the spatial position. Finally, a position sensitive ROI pooling layer is applied to aggregate the contents of the score maps to produce a new score for each ROI.

By applying this, significant improvements in object detections were found in powerful classification networks, obtaining state-of-the-art results in the PASCAL and COCO dataset with test speeds of 170ms [17, 19].

FPN

While R-FCN focused on the issue of translation invariance in object detection, Feature Pyramid Networks (FPN) focus on handling the issue of scale invariance. This is a major issue in terms of small object detection. Previous applications of feature pyramids required a significant amount of time and space in order provide an output.

FPNs merge two existing ideologies, single feature maps and pyramidal feature hierarchy. The former makes predictions based on single scale features which were used to decrease run-time and increase robustness to scale invariance, while sacrificing consistency between the training and test time inferences [17]. The latter allows for multi-scale feature representations which are semantically strong at all levels. However, introduces large semantic gaps due to the different depths.

The result of merging these ideologies is an architecture which provides several feature maps at different scales while still maintaining the semantic information of the features themselves in a fully convolutional manner. Moreover, this approach uses a bottom-up to generate the feature maps and top-down approach to provide *"higher resolution features by upsampling spatially coarser"* feature maps [20]. In addition, the features generated in the top-down section also communicates with the bottom-up layers to enhance them, making them semantically stronger.

The use of FPNs have proven to be very successful extracting rich semantic features from multiple levels. Moreover, it's versatility allows it to be implemented in various stages of a object detection architecture, like region proposals and even used in tasks such as instance segmentation.

Masked R-CNN

Masked R-CNN extends itself from Faster R-CNN by introducing an additional branch following the FC layer of the R-CNN to *"predict segmentation masks in a pixel to pixel manner"* [17]. This additional branch is applied to handle the challenge of Instance Segmentation and is made up of a small FCN.

The challenge is to detect all objects contained within an image, whilst also segmenting each instance of an object from each other [21]. This process of segmentation is called *semantic segmentation*, which attempts to classify the class of each pixel within an image to provide a bounding box that aligns itself with the object being classified.

The semantic segmentation is carried out using the additional mask branch to apply a $n \times n$ mask over the current ROI in parallel with bounding box regression and class prediction. Moreover, makes use of a RoIAlign layer rather than the RoIPool layer that is presented in [18] as Faster R-CNN was not made for pixel-to-pixel segmentation. The RoIAlign layer replaces the quantization of RoIPooling with a bilinear interpolation which can preserve the spatial location to increase mask accuracy.

5. PROPOSED METHODOLOGY

The research carried out it [17] compares the different architectures explained in the previous chapter and will form a basis for this experiment. Firstly, current state of the art object detection models will be obtained and trained using the Microsoft COCO dataset. Each model will undergo a base-line object detection test to prove the results seen in the initial experimental stage[17]. Once the models are evaluated, they will be fine-tuned to undergo object detection, making use of different target object sizes by varying distance from the target object. The best performing models are then used and applied to the Tello SDK drone for more vigorous testing.

The experimental area will consists of pre-defined distances accurately measured using measuring tape within a controlled space. By doing so, multiple variants can be controlled, such as the backgrounds, objects and reduces natural forces such as wind which can affect the stability of the drone.

Each model will be evaluated using the methodology explained in the following section. The first variation of the experiment requires a target object placed in a non-complex background. The second variation will alter the background to see the effects of a complex background on each model. Finally, for each form of background, the distance of the drone from the target object will be altered. By doing so, the amount of pixels classed to the target object are reduced in a natural manner.

6. EVALUATION

The results of the experiment will provide the correctness of each object detection model, referred to as *IoU (Intersection over Union)*. *IoU* is a ratio between the intersection and the union of predicted bounding boxes and ground truth boxes seen in *Equation 1*.

	Correct	Incorrect
Selected	tp	fp
Not Selected	fn	tn

$$IoU = \frac{Area_{Overlap}}{Area_{Union}} \quad (1)$$

The table above shows a confusion matrix to calculate the precision and recall of each detection model. *IoU* identifies the *True Positives*(T_p), *False Positives*(F_p), *True*

Negatives(T_n) and *False Negatives*(F_n) using a fine-tuned threshold ω .

$$Precision = \frac{T_p}{(T_P + F_P)} \quad (2)$$

$$Recall = \frac{T_p}{(T_P + F_N)} \quad (3)$$

Once these metrics are calculated, each model is applied to the experiment outlined in Section 5. The *maximum distances*, $Distance_{Max}$ respectively, for each model will be calculated manually using pre-defined distances within the experimental area. The new metric,

$$\Gamma$$

represents the ratio of average maximum distance for detection against the number of pixels classed to the target object.

$$\Gamma = \frac{AvgMaxDistance}{NumOfPixels} \quad (4)$$

These metrics will be calculated in two different cases:

- Object Detection in Simple Background
- Object Detection in Complex Background

7. CONCLUSION

The anticipated result of this Final Year Project provide a conclusive comparison between current state of the art models limitations of spatial information required. The research presented in this report provides a better understanding to the different architectures that each of these modules make use of. The application of the experiment outlined previously is next on the agenda. The implementation and evaluation carried out will follow what has been mentioned in sections 5 and 6 unless any issues arise.

8. REFERENCES

- [1] Y. LeCun, P. Haffner, L. Bottou, , and Y. Bengio, "Object recognition with gradient-based learning," *Shape, contour and grouping in computer vision*, vol. 38, p. 319–345, 1999.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Proc. 27th International Conference on Machine Learning*, pp. 1097–1105, 2012.
- [3] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," *Proceedings of the 25th International Conference on Neural Information Processing Systems*, 2010.
- [4] M. Lin, Q. Chen, and S. Yan, "Network in network," *International Conference on Learning Representations*, 2013.
- [5] S. Haykin, *Neural Networks and Learning Machines*. Hamilton, Ontario, Canada: Pearson, 3 ed., 2010.
- [6] A. Krogh, "What are artificial neural networks?," *Nature Biotechnology*, vol. 26, no. 2, pp. 195–197, 2008.
- [7] O. I. Abiodun, A. Jantan, A. E. Omolara, K. V. Dada, N. A. Mohamed, and H. Arshad, "State-of-the-art in artificial neural network applications: A survey," *Heliyon*, vol. 4, no. 11, p. e00938, 2018.
- [8] I. J. Goodfellow, Y. Bengio, and A. C. Courville, *Deep Learning: Adaptive computation and machine learning*. Cambridge, London, England: MIT Press, 2016.
- [9] S. Singaravel, J. Suykens, and P. Geyer, "Deep-learning neural-network architectures and methods: Using component-based models in building-design energy prediction," *Advanced Engineering Informatics*, vol. 38, pp. 81–90, 2018.
- [10] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning deep features for discriminative localization.," *Computer Vision and Pattern Recognition 2016*, 2016.
- [11] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," *Computer Vision and Pattern Recognition*, 2014.
- [12] J. Redmond, S. Divvila, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 779–788, 2015.
- [13] P. Felzenszwalb, R. Girshick, D. McAllester, and D. Ramanan, "Object detection with discriminatively trained part-based models," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, pp. 1627–1645, 2010.
- [14] X. Ren and D. Ramanan, "Histograms of sparse codes for object detection," *Computer Vision and Pattern Recognition*, 2013.
- [15] R. Girshick, "Fast r-cnn," *ICCV*, 2015.
- [16] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2014.
- [17] Z.-Q. Zhao, P. Zheng, S. tao Xu, and X. Wu, "Object detection with deep learning: A review," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 30, 2019.
- [18] S. Ren, K. He, R. Girshick, and J. Su, "Faster r-cnn: Towards real-time object detection with region proposal networks," *Computer Vision and Pattern Recognition*, 2016.
- [19] J. Dai, Y. Li, K. He, and J. Sun, "R-fcn: Object detection via region-based fully convolutional networks," *Computer Vision and Pattern Recognition*, 2016.
- [20] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," *Computer Vision and Pattern Recognition*, 2016.
- [21] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn," *Computer Vision and Pattern Recognition*, 2016.